

Budgie – Database Design

By Ella Huebotter

November 1, 2026

SWDV691

Introduction

I will be using PostgreSQL for my relational database. This will support the relationships in my data of:

- One user has many trips
- Each trip has many expenses and category limits

There are clear one-to-many relationships and relational foreign keys will support these nicely. It will also support frequent aggregations of totals (budget spent versus left), category percentages, and limit checks on categories and budget. These are simple SUM/GROUP BY queries in SQL. The data is also highly structured and organized.

Relationships

- User – trips (one to many)
- Trip – expenses (one to many)
- Trip – category limit (one to many)

Tables and Columns

User Table

The purpose of the user table is to store account and login information for each user. It also identifies each user for their trips

Column	Type	Notes
User_id	PK	Primary key
Email	VARCHAR(255)	Unique, used for login
Password_hash	VARCHAR(255)	Bcrypt hashed password
name	VARCHAR(100)	Display name

Example:

User_id	Email	Password_hash	name
1	ShaunaP@gmail.com	$2b$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy Bcrypt hashed password for 'MyTrip2026!'	Shauna

Trip Table

The purpose of the trip table is to store the users trips that they create with a title and total budget. This will be shown on the dashboard.

Column	Type	Notes
Trip_id	PK	Primary key
User_id	FK	Foreign key, references users (id)
Trip_title	VARCHAR(100)	e.g. "2026 Fall Trip"
Total_budget	NUMERIC(12,2)	Display name
Shared_user_id	FK	This is the shared users id stretch feature

Example:

Trip_id	User_id	Trip_title	Total_budget	Shared_user_id

1	1	2026 Fall Trip	2500.00	2

Category Limit Table

The purpose of the category limit table is to store the budget limits that the user sets on certain categories if they choose. One trip can have zero or more limits.

Column	Type	Notes
Category_limit_id	PK	Primary key
Trip_id	FK	Foreign key to identify the trip (id)
category	VARCHAR(50)	Category for the purchase e.g. Food, Transport, Shopping, etc.
limit_amount	NUMERIC(12,2)	Maximum budget limit for category

Example:

Category_limit_id	Trip_id	Category	Limit_amount
1	1	Food	500.00

Expense Table

The purpose of the expense table is to log all information associated with each purchase the user adds for a given trip.

Column	Type	Notes
Expense_id	PK	Primary key
Trip_id	FK	Foreign key to identify the trip(id)
Amount	NUMERIC(12,2)	The amount of the purchase (positive)
Category	VARCHAR(50)	Category for the purchase e.g. Food, Transport, Shopping, etc.
Title	VARCHAR(50)	Title e.g. Dinner with mom
Date	DATE	Date of purchase
Description	TEXT	Optional description such as “Dinner at outback with mom. Ordered steak and potatoes”
Added_by_user_id	FK	Stretch Feature to show which user added the expense

Example:

Expense_id	Trip_id	Amount	Category	Title	Date	description	Added_by_user_id

1	1	50.00	Food	Dinner with mom	10/27/2026	Dinner at Outback with mom. Ordered steak and potatoes	1

How the Data is Used

The dashboard will show the main business logic in the application. It will show how much money is left in the budget and how much has already been spent. It will show the chart with the category breakdown by percentage. It will also show a message if a category limit has been reached. The dashboard will show:

- Remaining budget = trips.total_budget − SUM(expenses.amount) for that trip
- Spent = SUM(expenses.amount)
- Percent used = (total spent / total budget) * 100
- Over budget = total spent > total budget
- Category pie chart = GROUP BY category with SUM(amount) and percentage of total spent: percent = (category sum / total spent) * 100
- Over limit alerts = compare SUM per category against category_limits.limit_amount

The 'My Trips' page list will allow the user to select a trip and it will show the title, budget spent, budget left, and percent used. This helps Nancy compare her trips. On the 'Add trip" page where the user can create a new trip or edit an existing one, the user will enter the information for the trip which is then stored in the database and used for dashboard calculations. The 'Add Expense' page will insert and log new expenses into the expenses table and then the dashboard will update with new data. The 'All expenses' page will let the user select an expense and if it is updated or deleted the dashboard will be queried again to update data.

The stretch feature will allow users to add a second user to a trip so they can also add or edit expenses and contribute. When logging an expense the user that adds the expense will be recorded in the database as 'added_by_user_id' so the user that added the expense can be shown on the All Expenses page. The budget and category limits will be the same for both users. The Shared_user_id will be set to null unless second user is added.

For deployment, Budgie will be hosted on Render. Render will give Budgie a PostgreSQL database and a database user to connect. The user will only have SELECT, INSERT, UPDATE, and DELETE rights to the four tables. Connection details will be stored in environment variables in Render.

Conclusion

PostgreSQL will be used for the Budgie app with the four tables: user, trip, category_limit, and expense. This will support the MVP and the three user personas. It contains one-to-many relationships and foreign keys. The data is highly structured and normalized making the relational database PostgreSQL a great fit.

Entity Relationship Diagram

